Manutenção e Evolução de Software

Programação Estratégica

Universidade do Minho

2025/2026

1 Generic Zippers

O Zipper é um mecanismo que permite definir movimentos e transformações numa estrutura de dados. Com um Zipper, basta indicar se se pretende navegar para uma de quatro direções (cima, baixo, esquerda, direita), sem ser necessário ter em conta qual a estrutura de dados em si que está a ser navegada, ou como é que esta está definida. A linguagem **Haskell** possui uma biblioteca para a criação de Zippers genéricos (i.e. não específicos para uma só estrutura de dados), de nome Scrap your Zippers[1].

Para a utilização desta biblioteca, deve-se converter a estrutura de dados a navegar para um Zipper com a função `toZipper`, seguido das eventuais operações a realizar no Zipper obtido, finalizando com a função `fromZipper` para obter a estrutura de dados original. Com um Zipper, pode-se navegar com as funções `up`, `down'`, `left` e `right`, sendo que essas funções poderão falhar (i.e. devolver `Nothing`) caso não seja possível navegar na direção pretendida.

Para a utilização desta biblioteca com um tipo de dados seu, deverá tornar esse tipo de dados uma instância da classe `Data`. Isto faz-se automaticamente, colocando a seguinte language flag no topo do ficheiro de código onde o tipo de dados está definido:

```
{-# LANGUAGE DeriveDataTypeable #-}
```

Será necessário também importar a biblioteca de `Data`, e indicar que os nossos tipos de dados devem derivar essa classe. A título de exemplo:

```
import Data.Data

data Foo = Foo deriving Data
data Bar = Bar Int
         | Counter String
         deriving (Show, Data)
```

[1] `https://hackage.haskell.org/package/syz-0.2.0.0`

2 Programação Estratégica

Programação estratégica envolve a travessia genérica de uma estrutura de dados, aplicando uma transformação ou redução a esta estrutura, codificando apenas as interações que interessam e indicando qual o tipo de travessia a ser feita. Vamos utilizar uma biblioteca desenvolvida no DI de nome Ztrategic, que internamente utiliza Zippers para a definição das travessias. Esta biblioteca encontra-se publicada no Hackage, pelo que poderá ser instalada ou descarregada com o uso de um package manager. Pode encontrar mais informações de uso, tutoriais e exemplos nas Lecture Notes disponibilizadas na Blackboard, ou em `https://bitbucket.org/zenunomacedo/ztrategic/`.

Para uma utilização mais streamlined desta biblioteca foi preparada uma pasta com uma configuração básica que usa o *stack* para tratar das dependências todas do projeto automaticamente - recomenda-se vivamente descarregar essa pasta da Blackboard e utilizar esta pasta para a resolução dos exercícios. O comando `stack ghci Example.hs` deverá carregar o ficheiro de exemplo com as dependências todas preparadas. Se não tem o *stack* instalado, recomenda-se a instalação do *ghcup* e a utilização desta ferramenta para instalar o *stack*.

Em alternativa, se não quer usar o *stack*, pode-se usar o *cabal* para instalar globalmente as dependências - nesse caso deverá ser necessário instalar como bibliotecas: *syb*, *syz*, *transformers*, *mtl*, *random*, *ZipperAG*, *monadplus* e *Ztrategic*.

Vamos considerar um exemplo muito básico de transformação de uma lista de inteiros. Para o uso da biblioteca *Ztrategic*, é necessário importá-la. Também é necessário importar *StrategicData* e definir a estrutura a atravessar como instância da classe *StrategicData*, que neste caso iremos fazer da forma mais básica. Atenção ao importar esta biblioteca, que ela redefine algumas funções de Zippers. Neste caso, podemos importar apenas a classe *StrategicData* pois o resto não será necessário.

```
-- Para o uso de Zippers
import Data.Data
import Data.Generics.Zipper
-- Para o uso de programação estratégica
import Language.Ztrategic
import Language.StrategicData (StrategicData)

instance StrategicData Int
instance StrategicData a => StrategicData [a]
```

Definimos `Int` como *StrategicData*, e definimos que se `a` é *StrategicData*, então `[a]` também é *StrategicData*. Caso tivessemos tipos de dados novos a ser atravessados por estratégias, deverá ser necessário torná-los *StrategicData*, da mesma forma que foi feito no exemplo acima para o tipo *Int*. Em caso de dúvida, o compilador irá reclamar se faltar esta declaração.

Vamos agora pegar numa dada lista de inteiros que será recebida como input da função, converter para um zipper, aplicar uma estratégia e voltar a converter para o formato original. Para isso usamos a função `applyTP` para aplicar uma estratégia TP (Type-Preserving), e vamos aplicar a estratégia `full_tdTP step`, em que `step` é o que queremos fazer em cada nodo e `full_tdTP` (full, top-down, type-preserving) transforma essa função para aplicar a toda a estrutura de dados, de uma forma topdown.

```
alteraListaS :: [Int] -> [Int]
alteraListaS lista =
    let listaZipper = toZipper lista
        Just listaNova = applyTP (full_tdTP step) listaZipper
              where step = idTP `adhocTP` alteraDois
    in
    fromZipper listaNova

alteraDois :: Int -> Maybe Int
alteraDois 2 = Just 12
alteraDois x = Just x
```

Repare na definição de `step`. Esta função tem por defeito o comportamento `idTP`, que é a função identidade, ao qual juntamos (usando `adhocTP`) a função `alteraDois`. Esta função `alteraDois` será aplicada indiscriminadamente a todos os nodos em que os tipos sejam compatíveis (neste caso, a todos os inteiros), e irá alterar todas as ocorrências do número 2 para 12, deixando todos os outros números inalterados. Note que a função `step` pode ser estendida com todos os tipos que forem necessários, podendo assim afetar estruturas de dados heterogéneas. Por exemplo, se se pretender alterar todos os números e todas as strings de uma determinada estrutura de dados, pode-se usar este `step`:

```
step = idTP `adhocTP` alteraNumero `adhocTP` alteraString
```

3 Exercícios

1. Analise o ficheiro `Let.hs` disponibilizado no projeto colocado na Black-Board. Adapte este ficheiro para conseguir usar programação estratégica sobre o tipo de dados Let. Note que isto implica ter a flag de `DeriveDataTypeable` no início do ficheiro, a derivação automática de `Data` dos seus tipos de dados e a declaração dos seus tipos de dados como instâncias da classe `StrategicData`.

2. Defina uma estratégia que percorra um programa `Let` e etiquete todas as variáveis com o prefixo "v_". Note que a função *step* deverá operar nos nodos da linguagem correspondentes a variáveis.

   (a) (Extra) Agora faça o mesmo mas para o tipo `String`. Dependendo de como o codificou, o código entrará num loop infinito ou repetirá a etiqueta duas vezes - explique porquê.

3. Defina estratégias semelhantes à do exercício 2. mas em vez de utilizar o combinador `full_tdTP`, experimente `full_buTP`, `once_tdTP` e `once_buTP`.

   (a) (Extra) As estratégias `once` não fizeram nada. Porquê? Tente-as corrigir após resolver o exercício 6.

4. Defina uma estratégia que percorra um programa `Let` e simplifique todas as ocorrências de $x+0$ para x, e $x*1$ para x, sendo que x representa qualquer expressão aritmética. Tenha atenção que não deve incluir chamadas recursivas no código que desenvolve, pois as estratégias tratam das travessias por si.

5. Defina uma estratégia que percorra um programa `Let`, modifique variáveis como no exercício 2 **mas apenas para atribuições** e simplifique expressões aritméticas como no exercício 4., tudo na mesma travessia.

6. A estratégia definida no exercício 4. não está totalmente correta. Identifique qual é o erro, escreva um contra-exemplo no qual as expressões aritméticas não fiquem totalmente simplificadas, e escreva uma estratégia que as simplifique corretamente. Para isto, deverá usar `innermost` em vez de `full_tdTP`. O combinador `innermost` aplica uma estratégia tantas vezes quantas forem possíveis aplicar numa estrutura de dados, até que essa estratégia deixe de fazer modificações na estrutura. Nesse sentido, a estratégia deve ser capaz de sinalizar que falhou - isso faz-se utilizando `failTP` em vez de `idTP` e devolvendo o valor `Nothing` em caso de não alteração do nodo nas funções de `step` que utilizarmos.

4 Estratégias Type-Unifying

As estratégias vistas até agora são do tipo type-preserving - em que se irá produzir um resultado do mesmo tipo de entrada. No exemplo acima, o input e o output são o mesmo, uma lista de inteiros. Vamos agora considerar estratégias type-unifying, em que é produzido um único resultado, essencialmente um *fold* sobre a estrutura de dados. Por simplicidade, vamos sempre produzir resultados do tipo `Maybe [a]`, mas o código permite o uso de outros monads desde que sejam instâncias de `Monoid` e de `MonadPlus`.

De seguida encontra-se um exemplo de uma estratégia que irá contar quantos números pares existem numa lista, como no exemplo mostrado acima:

```
contaPares :: [Int] -> Int
contaPares lista =
    let listaZipper = toZipper lista
        Just resultado = applyTU (full_tdTU step) listaZipper
              where step = failTU `adhocTU` contaPar
    in
    sum resultado
```

```
contaPar :: Int -> Maybe [Int]
contaPar n = if even n then Just [1] else Nothing
```

A aplicação da estratégia irá ser semelhante: a função `applyTU` irá aplicar a estratégia `(full_tdTU step)` a todos os nodos, que será a aplicação de `step` de forma full, topdown, type-unifying. O `step` é o default case de não fazer nada e falhar (`failTU`), que é conjugado (usando `adhocTU`) com a nossa função `contaPar`. A função `contaPar` irá devolver `Just [1]` se estiver a olhar para um número par e irá falhar com `Nothing` caso não esteja - também seria possível devolver o resultado 0.

Todas as estratégias type-unifying (usando estes monads) vão produzir uma lista de resultados - a nossa será uma lista de números 1, simbolizando as ocorrências de números pares. No final da função, somamos todos os números nessa lista para ter a contagem de pares.

5 Exercícios

1. Defina uma estratégia para contar os números pares de um programa na linguagem `Let`.

2. Defina uma estratégia para recolher todos os nomes de variáveis que ocorrem num programa na linguagem `Let`.

3. Defina uma função para a análise de nomes de um programa da sua linguagem, de acordo com os seguintes passos: Uma estratégia type-unifying irá recolher todos os nomes que são declarados na linguagem numa lista, e uma segunda estratégia type-unifying irá usar esse resultado nas suas funções step e irá verificar se todos os nomes usados se encontram nessa lista. Note que isto não é um algoritmo completo de análise de nomes pois não tem noção de scopes, mas já é capaz de detetar alguns erros num programa.